

NIGHTWATCH

Final hackathon implementation specification and critical design review

Agentic AI Hack - Tech Europe

19 September 2026 · Revision 2.0

Contents

Read this first	1
1. The review that changed the design	2
2. Scope, claims and non-goals	3
3. Architecture and trust boundaries	4
4. Deterministic application and payment model	5
5. Product viability and the technical thesis	6
5.1 Why this is worth building	6
5.2 Three technical features that make the demo defensible	7
5.3 Immediate harm limitation versus verified lease	7
5.4 Latency optimisation with proof preserved	7
6. Candidate creation and safe patching	8
7. Scenario and verification contract	9
8. Jev browser executor: implementable, not assumed	10
8.1 Choice of integration	10
8.2 Concrete observation/decision/execution loop	10
8.3 What Jev actually needs to prove	11
9. Control-plane contracts, state and APIs	12
9.1 Lease, staged checks and safe stop	13
10. Exact build plan and repository contract	14
10.1 Adapter contracts and bounded configuration	15
10.2 Control-plane orchestration pseudocode	16
11. Acceptance tests, preflight gates and failure modes	16
11.1 Tests that protect the thesis	16
11.2 Day-of smoke gate	17
11.3 Failure policy (no silent green state)	17
11.4 Definition of done and fallback levels	17
12. Demo presentation and what the result means	18
13. Source notes and verification limits	18

Read this first

Purpose. This is a build contract for an autonomous coding agent and the hackathon team. Implement the exact demo behavior and acceptance gates here. This is a controlled, fake-money prototype, **not** an authorization to autonomously deploy to a real production payment system. Where sponsor credentials or interfaces are unavailable, use the specified labeled fallback; do not invent an API or present a fallback as a live integration.

One-sentence product. NightWatch detects a post-deploy business correctness regression, replays it against isolated candidate mitigations, tests user journeys with Jev and independent payment-ledger invariants, then grants a temporary, scoped mitigation lease in a simulated rollout.

The crisp demo. A payment capture succeeds, its response is lost, and a newly deployed retry path uses a fresh idempotency key, producing two captures for one logical checkout. HTTP health stays green. NightWatch detects the duplicated charge, compares rollback, safe-handler feature flag and generated patch under the same seeded fault, selects the lowest-risk candidate that actually passes, then activates it for **future synthetic checkouts**. The originally harmed synthetic order remains visible for separate human reconciliation.

Success criterion. Target <180 seconds from IncidentEvent receipt to an active **demo** lease, measured with a monotonic clock and including reproduction, world startup, browser validation, independent oracle, rollout checks and receipt persistence. This is a measured target, not an SLA. If any mandatory gate fails or time expires, return ESCALATED with evidence; never quietly weaken tests to make the clock green.

1. The review that changed the design

The previous version had genuine strengths: a narrow incident class, independent checks, bounded mitigation choices, honest fallbacks and a clear receipt. Keep those. The following are the material defects and their exact resolutions.

Priority	Defect in v1	Resolution in this revision
P0	Disabling retry after a successful capture with a lost response cannot by itself confirm checkout. Yet the spec preselected the flag as winner.	Define an existing safe handler that performs a read-only provider lookup by persisted operation ID/key; the flag routes to it. An unknown result becomes <code>PENDING_CONFIRMATION</code> ; no second capture. Never preselect the winner.
P0	Candidate code could influence an in-world payment stub or app-owned evidence endpoint.	Put the fake provider's append-only ledger and the evaluator outside the candidate world. Evaluator queries provider directly and compares frozen input with app-visible status.
P0	One capture per order misses two orders from one double-click. Money was represented by decimal floats.	Give each logical checkout an immutable <code>checkout_intent_id</code> ; enforce one intent to one order to one capture to one confirmation. Use integer pence.
P0	Original post-incident data could contaminate replay. Unique order IDs alone do not isolate refunds/inventory/faults.	Preserve the original incident ledger separately. Clone a pre-transaction fixture for each candidate and reset per scenario with namespaced provider ledger and fault schedule; only one scenario mutates each world at a time.

Priority	Defect in v1	Resolution in this revision
P1	The Jev example is synchronous and attaches to local Chrome; asyncio does not create browser isolation. Fixed confidence thresholds were treated as correctness probabilities.	Use three prewarmed isolated browser worker processes, one per candidate; run scenarios serially per world. Test true concurrency. Check both operation and target margins, calibrate on labeled traces, and fail closed on uncertain mutation.
P1	The 5/25/100% “canary” had too few synthetic transactions for statistical claims; revocation could restore the known bug.	Name it a simulated staged rollout smoke check, list probe counts and deterministic gates. Stop/expiry freezes safe routing or moves to safe hold; never auto reenable the bad retry path.
P1	Fixed scripted checkout would make Jev look decorative; existing AI SRE tools already correlate incidents and deploys.	Test two semantic UI variants plus an impact-derived journey; compare against scripted Playwright and an LLM browser executor if available. Claim the combination and measured result, not first-ever AI SRE.
P2	Eight mandatory scenarios, 8-12 generated scenarios, and “two or three” required worlds conflicted.	Exactly eight mandatory scenario IDs, exactly three candidate worlds when the full demo is claimed. Two-world fallback is explicitly a reduced demo. Gemini may add optional cases only outside the gate.

The payment recovery logic follows the general principle in [Stripe’s idempotency and network-error guidance](#): uncertain mutation outcomes cannot safely be retried with a fresh key. This project uses an **independently specified fake provider**, not a claim that Stripe exposes our stub’s lookup endpoint.

2. Scope, claims and non-goals

Supported: a recent deployment/config change causes a reproducible semantic violation in a synthetic web checkout; candidate actions are known-good release rollback, a narrow feature flag to a safe handler, and a bounded payment-module patch. The incident is qualified by an invariant event and comparison on identical pre-transaction data.

Not supported: arbitrary incident diagnosis, real provider charges/refunds, shell access to production, autonomous migrations, secrets/IAM changes, permanent code merge, refund of an already affected customer, or unmeasured “instant” global safety. A safe-looking matched replay is evidence for this fixture, not a proof of universal root cause or production readiness.

What judges can see: (1) original duplicate capture in an independent ledger, (2) health endpoint still 200, (3) three live candidate worlds and timestamps, (4) real Jev action/target selections and browser traces, (5) a negative control rejected by an independent invariant, (6) a selected and scoped synthetic mitigation lease, (7) a receipt that still lists the original affected order.

Differentiation, stated narrowly. Existing services already correlate deploys, telemetry, code and incidents or guard feature rollouts. NightWatch’s hackathon hypothesis combines **business-invariant-triggered detection + matched runnable mitigation futures + adaptive user-journey attack + independent ledger oracle + scoped temporary simulated actuation**. Do

not claim any one of these pieces is unprecedented. [Rootly](#), [incident.io](#), [Datadog](#) and [LaunchDarkly](#) are adjacent work.

3. Architecture and trust boundaries

1. **Trusted control plane:** FastAPI/Python 3.12, incident state machine, event store, scenario scheduler, Gemini and Conduct adapters, candidate selector, lease router and receipt. This owns every secret and the evaluator source.
2. **Trusted fake provider:** one local/service process with separate append-only SQLite store, immutable capture/refund events, operation-idempotency index and per-scenario fault namespace. Candidate receives only a fake scoped token for its namespace. Evaluator queries the ledger through a separate control credential. Do not persist the trusted ledger inside a patchable candidate image. Preflight reachability from Modal; if outbound networking cannot be restricted appropriately, use a **pretested** local Docker topology with an internal provider network and still label the execution mode.
3. **Untrusted candidate worlds:** exactly three Modal Sandboxes, each with isolated app code, app SQLite DB, flag config and a world-local FastAPI process. Only the payment retry module is patchable. Its exact allowlisted path is given in Section 6. App state is evidence, never the sole authority about captures.
4. **Trusted browser workers:** one prewarmed process and throwaway Chromium profile/context per candidate. A worker owns the browser session and submits each scenario's fixed input values. A new context, cookies and storage are created for each scenario. All actions are bounded to its candidate URL.
5. **Dashboard:** static React/Vite build served by control plane; SSE for durable timeline and status. A disconnected dashboard cannot cancel verification or create a lease.

The candidate talks to the fake provider over a scoped endpoint; evaluator independently reads that provider ledger and the candidate's app evidence. Provider side effects carry a namespace (`incident_id`, `candidate_id`, `scenario_id`) and immutable `operation_id`. All worlds use the **same frozen scenario input and one-shot fault schedule**, in separate namespaces. The detector's original ledger uses namespace `original` and is never reset by replay.

For Modal: HTTPS encrypted ports alone are not authorization. Use [Sandbox Connect Tokens](#) for control/evidence traffic or a proxy with equivalent per-world authorization; never expose unauthenticated `/internal/*` endpoints. Avoid putting token query parameters in screenshots/logs. Test connection, timeout, cleanup and selected network policy before the hackathon. No real production secrets enter a candidate.

3.1 Concrete deployment topology

Boundary	Owns	Connects to
Trusted control plane	State, candidate policy, oracle, leases and signed evidence	Fake provider ledger and each authorized candidate world
Trusted fake provider	Namespaced append-only captures, refunds and fault schedule	Candidate calls with scoped fake token; evaluator reads with separate credential
Modal worlds A/B/C	Three independently runnable app versions and app SQLite databases	Scoped provider endpoint and one owned browser worker each
Browser workers A/B/C	Disposable Chromium profile/context and Jev decisions	Only their assigned authenticated world URL; traces to control plane

Failure boundary: if true external provider separation is not ready, do not grant an automatic lease based on app-owned receipts. A local Docker fake provider on a separate trusted process/storage is an acceptable **labeled** build fallback; pretending an in-app stub is independent is not.

4. Deterministic application and payment model

4.1 Frozen domain objects

Use exact names and fields below in `demo_store/domain/models.py` and SQLite migrations established before the live run. All currency amounts are **integer minor units**; use GBP in the fixture and format pounds only in UI.

- `CheckoutIntent(intent_id, customer_id, cart_hash, created_at, status)`; unique (`customer_id, intent_id`). Intent is generated once before the first submit and retained through double click/refresh.
- `Order(order_id, intent_id UNIQUE, amount_minor, currency, items_json, status, operation_id UNIQUE, idempotency_key UNIQUE, confirmation_id UNIQUE NULL, version)`; frozen amount/currency/cart hash once payment starts. Order statuses: `CREATED, PAYMENT_IN_FLIGHT, PENDING_CONFIRMATION, PAID, DECLINED, REFUNDED_PARTIAL, REFUNDED_FULL`.
- `PaymentOperation(operation_id, order_id UNIQUE, key, frozen_amount_minor, currency, created_at)`. The stable key is derived once per operation (e.g. `demo-capture:{operation_id}`) and persisted **before** the provider call. Do not use the same key with changed parameters. A new cart creates a new intent/order/operation.
- `ProviderCapture(capture_id, namespace, operation_id, key, amount_minor, currency, status, created_at)` in the trusted provider only; (`namespace, key`) uniqueness returns the cached capture for identical parameters. Parameter mismatch returns 409 without a new capture. The provider keeps the key for the whole demo and does not model retention expiry.
- `ProviderRefund(refund_id, namespace, capture_id, refund_intent_id, amount_minor, created_at)`; unique (`namespace, refund_intent_id`); total refunded never exceeds captured amount.
- `FaultPlan(namespace, scenario_id, phase, remaining)`; `DROP_AFTER_CAPTURE_ONCE, TIMEOUT_BEFORE_CAPTURE_ONCE, DECLINE, NONE`. Creating a namespace reinitializes this plan; one world cannot consume another world's fault.
- `Confirmation(confirmation_id, intent_id UNIQUE, order_id UNIQUE, displayed_at)`. Server returns same confirmation on client replay; UI may render it twice, but no second confirmation entity is created.

4.2 Exact provider protocol

Define a typed fake provider client in `demo_store/domain/provider_client.py`; implement trusted server in `trusted_provider/`. All requests carry namespace and fake scoped token. `POST /capture` takes (`operation_id, key, amount_minor, currency`), atomically indexes (`namespace, key`) and appends at most one capture **for that key**; for a new key it appends another capture even if `operation_id` repeats, deliberately reproducing the bug. `GET /operations/{operation_id}/captures` returns an authorized read-only list (the safe handler uses this to reconcile). `POST /refund` enforces total and idempotency. `GET /internal/ledger/{namespace}` requires evaluator credential and returns captures/refunds/fault events. Candidate cannot call evaluator endpoint.

A `DROP_AFTER_CAPTURE_ONCE` request writes a successful capture, then throws a simulated timeout **without returning a capture ID to the app**. `TIMEOUT_BEFORE_CAPTURE_ONCE` throws without a capture. On a subsequent request with the same key and frozen parameters the provider returns the already stored result; with a fresh key it captures again. Never infer a successful capture solely from a timeout.

Known-good safe handler, present in previous release and behind the flag in the current release: persist operation/key; call provider once; on timeout query captures by operation ID. If exactly one matching (`amount_minor, currency`) is present, transition to `PAID` and create/reuse the unique confirmation without another capture. If zero or lookup fails, transition to `PENDING_CONFIRMATION`, show an honest pending screen and schedule controlled reconciliation; **do not create a new capture while outcome is unknown**. If more than one capture or mismatched parameters, raise incident and quarantine intent. Baseline only guarantees immediate success for the controlled `DROP_AFTER_CAPTURE_ONCE` case when lookup responds; it does not falsely claim availability for every network failure.

Buggy new handler: on initial call and timeout retry, creates a new random key on each attempt.

For `DROP_AFTER_CAPTURE_ONCE`, two successful captures occur and a success response may be returned. Capture ledger, order state and confirmation reveal the violation. The bug is confined to `demo_store/domain/payment_retry.py`.

Generated patch target: a minimal change to that one file that reuses the persisted operation/key and inquires after an uncertain timeout. If a matching capture is present, settle; if none is present, return `PENDING_CONFIRMATION` with no automatic second capture. A future retry is allowed only after a definitive, separately authorized reconciliation decision and must reuse the operation/key and frozen parameters. Even a passing patch remains a suggested permanent fix, not auto merged.

Feature flag B: `payment_retry_handler=safe` routes to the known-good reconciliation handler. Call it `route_to_safe_handler`, not `disable_retry`. It changes only this traffic path. **Rollback A** uses the previous known-good release, including the same safe handler. **Patch C** keeps the new release but fixes its retry code. Candidate ranking is policy-driven after tests; B is expected to win for this fixture only if it passes and has lower configured disruption than A/C.

4.3 Transition and invariant rules

App payment flow: `CREATED -> PAYMENT_IN_FLIGHT -> PAID | DECLINED | PENDING_CONFIRMATION`; pending can move to `PAID/DECLINED` only after trusted reconciliation evidence. A paid order can become `REFUNDED_PARTIAL` or `REFUNDED_FULL`, without erasing the historical capture. Refunds are separate events; original charges remain visible.

Independent oracle checks after each scenario; all outputs are typed `InvariantResult` with raw ledger and app evidence identifiers, never just an LLM verdict:

- **INV-01 / always:** for each logical intent, at most one order, at most one successful capture across all its operations/keys, and at most one confirmation entity. A failed or pending intent may have zero captures; an unresolved captured intent may have one but cannot be called successful.
- **INV-02 / settled:** `PAID` or `REFUNDED_*` implies exactly one successful capture; amount and currency match the frozen order; one confirmation exists. A `DECLINED` order has zero captures and no confirmation. `PENDING_CONFIRMATION` has no success confirmation, even if provider holds an uncertain capture; it is a truthful degraded state rather than a success.
- **INV-03 / money:** all minor-unit amounts positive for captures and refunds; `sum(refunds) <= capture.amount_minor`. Full/partial refund statuses agree with refund sum, and repeated refund intent never appends a second refund.
- **INV-04 / fulfillment:** inventory decreases exactly once for each fulfilled order; an incomplete/declined order does not fulfill. For the eight mandatory scenarios, enforce this whenever inventory is touched.
- **INV-05 / service:** checkout route stays available and the user receives a correct `PAID` confirmation in every scenario that expects settlement; do not accept a 200 with no capture or a hidden `PENDING_CONFIRMATION` as equivalent success.
- **INV-06 / historical separation:** original affected order and duplicate captures remain visible in receipt; candidate fixture starts before that transaction and cannot modify original ledger.

For declined/pre-capture timeout scenarios, require zero captures and truthful pending/declined status as specified in the scenario table. Do not apply a global `capture_count == 1` to every order or global `PAID iff captured after refunds`.

5. Product viability and the technical thesis

5.1 Why this is worth building

A company can have green infrastructure health while a deploy breaks money or entitlement correctness. Existing APM and incident tools can identify suspicious changes, but a decision to roll back a release or flip a feature flag depends on whether the alternative actually preserves the **business invariant** and the user journey. NightWatch is valuable when it can compare several runnable mitigations against the same realistic starting state and surface an auditable recommendation before the incident team finishes manual triage.

The strongest commercial wedge is **release verification for known critical workflows**: teams register a small invariant pack and disposable data fixture, connect deploy IDs/flags, then use

NightWatch in preproduction or shadow traffic first. The next level is human-approved, production-adjacent mitigations. Only a preauthorized and reversible safe handler/hold should run automatically. Full autonomous payment changes, live DB cloning or arbitrary generated patches are outside a plausible first deployment. This changes the pitch from “AI can fix anything in three minutes” to “verified, scoped containment for workflows with explicit correctness contracts.”

Adoption prerequisites: a replayable transaction fixture, trustworthy provider/ledger observation, known-good mitigation or safe hold, isolated test environment, a policy owner, and a route to apply/revoke an approved feature flag. If a customer cannot supply these, NightWatch offers investigation evidence only, with no autonomous lease. This is a meaningful product boundary, not a feature silently faked in the demo.

Operational cost: for the hackathon, three short-lived sandboxes and browser workers are justified by the live tournament; in production, cap worlds and time per incident, cache base images, reuse warm capacity, and early-reject unsafe candidates. Keep a per-run budget in the receipt. Do not promise a cost figure before measuring it.

5.2 Three technical features that make the demo defensible

A. Evidence-first counterfactual comparison. The control plane freezes a pre-transaction fixture hash, deploy identity, generated fault schedule and oracle version. It first proves the bad release fails; then runs candidate A/B/C against equivalent, isolated worlds. Record `bad_failure_signature` and each candidate’s ledger outcome. This demonstrates mitigation effectiveness for a controlled counterfactual without overstating proof of root cause.

B. Adaptive user-journey testing. Jev selects an operation plus a target from currently observed semantic controls, while scenario code supplies known input values. The page has two controlled UI variants: version X uses explicit “Pay now” and a normal promo field; version Y reorders fields, calls the control “Confirm order”, and reveals promo input after a semantic “Add voucher” control. Variants preserve the same backend contract and oracle. At least one mandatory browser journey must run on an executor-unknown variant chosen from a deterministic seed; otherwise a fixed Playwright script is sufficient and Jev contributes little. Compare against scripted Playwright honestly.

C. Proof-carrying temporary mitigation. A lease references the exact candidate code/flag hash, fixture/oracle versions, scenario results, browser traces and rollout probes. The router only accepts a passing evaluation certificate signed/issued by trusted control-plane code. A model cannot issue or extend a lease. Afterward a guardian performs synthetic ledger checks; uncertainty moves routing into safe hold and pages a human, never back into the known-bad path.

These are integration design hypotheses demonstrated in a controlled system, not claims of unprecedented algorithms or a production-ready payment SRE.

5.3 Immediate harm limitation versus verified lease

On observing duplicate capture, the trusted router activates a **preauthorized emergency SAFE_HOLD** for subsequent affected intents: it blocks new capture submissions for the affected cohort and displays a temporary checkout-unavailable state before money moves; for already in-flight uncertain orders it permits read-only provider inquiry and displays `PENDING_CONFIRMATION` until resolved. Record `hold_started_at` separately from `lease_issued_at`. This action is defined and tested before the hackathon; Gemini does not choose it. Affected transactions already in flight may still complete badly, so the dashboard never claims zero further harm from the instant of detection.

The arena then tests safe-handler flag, rollback and patch. A passing candidate can replace `SAFE_HOLD` through staged synthetic checks. If all candidates fail, remain in `SAFE_HOLD` and `ESCALATE`. This architecture avoids leaving the known bad path active for the entire 180-second investigation and makes the containment story credible. Its tradeoff is temporary checkout degradation for uncertain outcomes; display it explicitly. In a real integration, a company must opt into this exact policy and own customer communication.

5.4 Latency optimisation with proof preserved

1. **Precompute before the incident:** build image/UI, seed fixture and world templates, validate provider network, warm three browser workers, pin model, load invariant pack and known-good

deployment. These preparatory times are outside detect-to-lease but reported separately; do not suggest a cold installation takes <180 s.

2. **Start independently available work together:** parse deploy diff and obtain Conduct graph while reproducing bad fixture; begin A/B world creation from known metadata while Gemini drafts C. Do not let optional Conduct block reproduction.
3. **Separate cheap proofs from adaptive UI:** run deterministic provider/API assertions first; reject a candidate immediately on duplicate capture, fixture mismatch or build failure. Use two mandatory adaptive browser journeys per passing candidate; remaining mandatory cases are deterministic API flows.
4. **One browser per world:** three simultaneous worker processes, serial scenarios within each world. This yields actual concurrency of at most three, avoids DB resetting races, and makes resource use predictable. Measure peak active workers, queue and API limits. Higher scenario concurrency is a stretch gated by an isolation test.
5. **Policy-monotonic early decision:** B has risk rank 0. Once B passes every mandatory gate and synthetic rollout, a later A/C result cannot outrank it. A lease may be issued while higher-risk candidate evaluation completes in the background, but the receipt must mark those IN_PROGRESS and update it when they finish. For the full three-world showcase, wait until all three are evaluated if doing so still meets the target. Never describe untested C as passing.
6. **Hard deadlines:** stage timeouts have explicit failure states; no timeout turns into PASS. Measure full observed p50/p95 over rehearsals, not a budget guessed from a vendor's microbenchmark. If the fast path cannot complete, display the true time and a safe hold/escalation outcome.

5.4.1 Critical-path budget, engineering target only

- **0-3 s:** preserve the original evidence and activate preauthorized SAFE_HOLD. Gate: router readback and durable incident event.
- **3-15 s:** reproduce the bad release while A/B worlds start and diff/Conduct work begins. Gate: same duplicate signature, fixture hash and fault seed.
- **12-35 s:** obtain Gemini's structured patch plan while A/B run early API checks. Invalid C must not block A/B.
- **25-55 s:** build C, run A/B deterministic tests and ready three browser workers. Gate: app health and isolated provider namespaces.
- **45-115 s:** run two Jev journeys per eligible candidate, six API cases and independent oracle checks. Gate: all eight mandatory cases for the selected candidate.
- **115-160 s:** select and run synthetic staged smoke checks. Gate: no invariant failure or known-bad route.
- **160-180 s:** atomically issue and read back the lease, persist receipt and show stopwatch. Higher-risk C may finish later with truthful IN_PROGRESS status.

These intervals overlap; they are not durations to add. A slow Gemini/Modal/Jev call can violate them. Include startup, initial navigation, oracle and receipt in the observed total. Model latency distribution, not an advertised p50, determines headroom. Before the event, run at least two end-to-end rehearsals after the final material code change.

6. Candidate creation and safe patching

- **B · safe-handler flag (risk rank 0).** Current release with the allowlisted safe-handler flag enabled. On a post-capture lost response, the existing safe handler inquires by operation ID, returns PAID with exactly one capture and one confirmation. It changes only the affected checkout path and is the narrowest reversible action.
- **A · rollback (risk rank 1).** Run the known-good release hash, including the safe reconciliation handler. Expect the same payment correctness, while unrelated release changes are also removed.
- **C · generated patch (risk rank 2).** Current release plus Gemini diff restricted to demo_store/domain/payment_retry.py. Persist/reuse operation key and inquire after timeout; unknown outcomes remain PENDING without automatic recapture. It is new code and needs permanent human review.

The selector considers only **eligible** candidates: build checks pass; bad failure no longer reproduces under matched input; all eight named scenarios terminate and pass their declared invari-

ants; no forbidden provider side effect; at least one adaptive UI variant passes; route stays usable; candidate hash matches evaluated hash. If ties remain, choose (rank, changed_file_count, candidate_id) deterministically. Do not hard-code B as a winner or create a fake failure in A/C.

Gemini receives redacted Incident Capsule, diff, allowed file path, provider contract summary, impact graph and invariant IDs. It produces one typed GeminiPlan(diagnosis, mechanism, patch_diff, touched_files, suggested_extra_scenarios, explanation). Its scenarios are optional probes; it cannot remove or rewrite the eight mandatory ones. If Gemini fails, C is SKIPPED_GENERATION, A/B still run. It never returns a lease decision.

Patch validation: parse unified diff; normalize and verify every path is the one allowed file; reject symlinks, mode changes, binary patches, imports of trusted_provider/evaluator internals, modifications to seeds/config/tests, more than 40 changed lines, dependency changes or arbitrary shell commands. Apply to a clean candidate checkout using `git apply --check, apply, run ruff, syntax check` and focused tests. These checks are necessary but do not establish safety; the independent ledger/invariant suite is authoritative. Store patch hash. Candidate receives only fake, namespaced provider credential. On any uncertainty, mark C failed rather than relaxing patch policy.

Negative control for evaluator integrity: in a test-only world, return HTTP 200 and “paid” UI without calling provider. It must fail INV-02/INV-05. Another test creates two orders from the same intent, each with one capture; it must fail INV-01. Never display a negative control as a real generated candidate.

7. Scenario and verification contract

There are **exactly eight mandatory scenario IDs**, not five plus eight generated. Use identical inputs/faults across A/B/C in distinct namespaces. UI means Jev must traverse a semantic browser form; API is a deterministic transaction call with the same trusted oracle. Mandatory per-candidate execution is two UI + six API scenarios. UI variants are determined from a fixed run seed unknown to browser policy; the receipt records which variant ran. The extra UI comparison benchmark is outside the containment critical path.

Mandatory scenarios. The parenthesized IDs are the required independent checks; a missing result fails that candidate.

- **S01 · UI / variant X / normal payment.** One submit, one SKU-A. Expect PAID, exactly one order, capture and confirmation, inventory 99. Check INV-01, INV-02, INV-04, INV-05.
- **S02 · UI / variant Y / lost response after first capture.** Expect PAID after provider inquiry or a safe same-key recovery, with exactly one capture and one confirmation. Check INV-01, INV-02, INV-05.
- **S03 · API / concurrent double submit.** Send two simultaneous requests for the same intent. Expect the same order and one capture/confirmation. Check INV-01, INV-02, INV-05.
- **S04 · API / refresh and replay.** Submit a paid intent again. Expect the original order and confirmation ID with no fresh capture. Check INV-01, INV-02.
- **S05 · API / quantity two and promotion, lost response after capture.** Frozen total 14,998 pence, one capture, inventory 98. Check INV-01, INV-02, INV-04.
- **S06 · API / full refund and duplicate refund request.** Exactly one refund for 7,999 pence, historical capture preserved, status REFUNDED_FULL. Check INV-01, INV-02, INV-03.
- **S07 · API / partial refunds and replay.** Refund 2,000 then 1,000 pence using distinct refund intent IDs; replay the second ID. Aggregate stays 3,000 pence, status REFUNDED_PARTIAL. Check INV-01, INV-02, INV-03.
- **S08 · API / timeout before capture.** Provider inquiry finds no capture. Expect PENDING_CONFIRMATION, zero captures and no confirmation; no blind retry. Check INV-01, INV-02, INV-05 (the truthful pending branch of INV-05).

All world/scenario sequences begin by resetting the candidate app DB from **pre-transaction seed** and creating a new trusted provider namespace with the declared fault. When the app is quiescent, copy SQLite with the [SQLite Backup API](#) or recreate from schema/seed transaction; do not cp a live WAL database. Assert seed hash, code hash and provider fault plan before requests. Open a new browser context, run scenario, wait for a bounded stable outcome, query provider ledger as evaluator, inspect app state, evaluate checks, persist raw evidence and trace, close context, then move to next scenario. A failed reset/namespace/trace makes the scenario FAIL and candidate ineligible.

S08 is a **safety outcome**, not a successful paid checkout: it passes only when the customer is told the outcome is pending and the fake provider has no capture. If lookup is unavailable after S02, PENDING_CONFIRMATION can be safe but fails S02's availability requirement; B must not win based on safety alone. For S03, app transaction locking or unique constraints must handle two concurrent requests without two orders. For S06/S07, refund must never delete the capture record.

Fixed fixture values. Each scenario begins with a new customer/intent, empty order/provider namespace and stock of 100 units of SKU-A. The normal cart is one SKU-A at 7,999 pence; S05 uses two SKU-A (15,998 pence) and a fixed 1,000-pence promotion, so the frozen total is 14,998 pence. S06 refunds 7,999 pence; S07 refunds 2,000 then 1,000 pence and replays the second refund intent (total remains 3,000). S01 stock ends at 99; S05 stock ends at 98; unsuccessful S08 stock stays at 100. UI fixture text is static per scenario and not generated by Jev. All IDs and fault events include candidate/scenario namespace.

Reproduction gate. The original incident lives in namespace original; create a fresh namespace repro-bad from frozen pre-transaction fixture and replay S02 on the buggy release. It must produce exactly two captures with distinct keys, a matched amount and a normal-looking successful checkout, while /health is 200. If that signature is absent, ESCALATE, do not mint lease. Reproduction replay does not alter the original harmed order.

Conduct-directed tests. When a real Conduct interface is supplied, map changed file -> service -> business process, and use the result to choose the optional third Jev journey (e.g. refunds or confirmation) from a trusted scenario registry. It cannot manufacture new mandatory assertions or weaken them. If Conduct is unavailable, read fixtures/impact_graph_fallback.json, label source STATIC_FALLBACK, and run the same registry. No guessed Conduct endpoints or "live Conduct" UI badge.

8. Jev browser executor: implementable, not assumed

8.1 Choice of integration

TypeSafe describes Jev as an early-access fast, typed-decision model. The [official SDK has an async client](#), and Pydantic AI has a [TypeSafe integration](#). The current [browser-use/jev-ultrafast](#) is an excellent **reference** for compact DOM snapshots, indexed actions, stale guards and independent verification; its published Agent.run() is a synchronous generator using local Browser Harness/Chrome. Do not call it as if it were a ready-made asyncio multi-browser executor. Its owned tabs share the connected Chrome profile unless adapted.

Hackathon implementation decision: build JevExecutor with async_playwright for observation/execution and AsyncTypeSafeClient for bounded decisions. Prewarm three separate OS processes or browser instances; each process owns one ephemeral Chromium user data directory, one world URL and serial scenario queue. Each scenario creates a fresh browser context/page; no personal Chrome/debug profile. The control plane awaits workers via RPC/queues; a process crash fails that scenario. Preflight two simultaneous cross-world Jev journeys and verify distinct cookies, intent IDs, provider namespaces and browser profile paths before claiming parallelism. If async port misses its smoke gate, use a pretested synchronous jev-ultrafast worker **process per world**, with dedicated CDP profile/endpoint and no shared process event loop. If only one worker is reliable, label the demo serial and do not claim parallel browsers.

Do not make CUA Fleet mandatory. CUA Driver's bounded action and capture-binding pattern is useful for future non-DOM apps, but adds OS permissions and another browser stack. Keep BrowserExecutor as the adapter boundary for a later CUA implementation. No CUA integration counts toward the core acceptance gate.

8.2 Concrete observation/decision/execution loop

Use native HTML buttons, inputs, labels, options and ARIA roles in both UI variants. Build a compact snapshot containing url, title, visible semantic controls with stable generated action_id, role, accessible name, current value, disabled/hidden state, and up to three preceding action outcomes. Strip customer private text, patch contents, oracle results and secrets. If initial snapshot has no checkout controls, wait at most 2 seconds for DOM readiness and reobserve, then fail. Cap the candidate list; [Pydantic's Jev guide](#) documents a 255-option ceiling for a single choice. Filter invisible/incompatible controls before asking.

Ask a Jev Choice over allowed operations (CLICK, TYPE_TEXT, SELECT, WAIT, REOBSERVE, ABSTAIN, DONE), with compatible target choices in parallel when the SDK pattern supports it. The target choices contain only IDs observed in this snapshot. The official [TypeSafe pattern](#) explains speculative independent questions. Keep each decision head atomic: operation selection is separate from compatible target selection. Store both the selected value and **per-head confidence margin**. DONE is an agent control decision only; never a verification result.

For TYPE_TEXT, lookup (scenario_id, field_semantic_key) in the scenario's fixed fixture values and type the validated value. Jev never invents payment amounts, customer details, selectors, coordinates or shell/code. A missing semantic key fails the action. A real application may require a separately configured text helper; the hackathon fixed fixture needs none.

Before executing, recheck URL origin, page generation, target locator identity, visibility/enabled state, hit testing and current form context. If changed, discard decision and reobserve. After a mutation, wait for a bounded observable postcondition and take a new snapshot. Never repeat an uncertain submit/capture mutation blindly; query app/provider state or move to ABSTAIN. Stop at 20 steps/25 seconds per UI scenario, with an independent watchdog and no hidden retry budget.

Pseudocode; adapt field names after the SDK smoke test.

```
async def step(snapshot, scenario):
    choices = build_compatible_choices(snapshot) # only observed action IDs
    reply = await jev_choose(snapshot.compact_state, choices)
    op = reply.operation
    target = reply.target_for(op) if op.needs_target else None
    invalid_op = op not in choices.operations
    invalid_target = target and target not in choices.targets[op]
    if invalid_op or invalid_target:
        return ABSTAIN
    if not calibrated_gate(op, reply.operation_margin, reply.target_margin):
        return await reobserve_once_or_abstain(snapshot, op)
    if not await still_current(snapshot, op, target):
        return REOBSERVE
    if op == TYPE_TEXT:
        value = scenario.fixture_fields.get(choices.semantic_key(target))
        if value is None: return ABSTAIN
        await execute_bounded(op, target, value)
    elif op in {CLICK, SELECT}:
        await execute_bounded(op, target)
    elif op == DONE:
        return DONE_UNVERIFIED
    return await observe_new_snapshot()
```

Margin policy. Pydantic explicitly says provider_details['confidence'] is a **margin, not a probability the answer is right**. There is no universal meaningful 0.80 = 80% safe. During preparation, label operation/target outcomes on at least 30 pilot decisions as a smoke check, then tune independent per-operation gates (especially SUBMIT and mutation vs navigation); record errors, abstentions and fallback rates. Thirty decisions cannot establish a low failure probability. At event start, pin the Jev version against which gates were tested (currently documented example jev-1.13.0); fail preflight if actual model differs. The default conservative rule for an uncalibrated mutation is REOBSERVE/ABSTAIN and deterministic fallback for that entire scenario, **labeled as not Jev-tested**. A fallback cannot be counted as a Jev win.

8.3 What Jev actually needs to prove

Jev's value is adaptive navigation under modest UI changes, not doing arithmetic or taking over the payment safety decision. Implement the same user goal for both UI variants without a variant-specific selector table in Jev. Run a benchmark on identical fresh fixtures for:

1. Site-specific scripted Playwright (expected strong speed floor on a fixed checkout).
2. Jev adaptive executor (the production-like adaptive option).
3. Conventional LLM browser executor **only if available and configured before the event**; absence is NOT_RUN, never an invented number.

Alternate arm order for at least three repeats per arm/variant, same fault and fixture, verify outcome

with the same independent oracle. Report attempts, success count, end-to-end median, model decision median, worker/browser setup, initial navigation, oracle time, token/call count, fallback rate and actual peak parallel sessions. A three-repeat demonstration is descriptive, not statistically decisive. No claim Jev is universally faster than Playwright or an LLM. The author's [7.073-second Jev example](#) excludes setup/navigation/independent post-run checking and its three paired comparisons are optimized-vs-original Jev runtime, not Jev-vs-frontier LLM.

9. Control-plane contracts, state and APIs

Use Pydantic v2 models with validated enums and extra="forbid". Give mutable fields factory defaults. IDs are UUIDs or deterministic fixture IDs with a namespace. Persist UTC wall time as well as monotonic elapsed time; a monotonic value alone cannot be interpreted after restart.

```
class IncidentEvent(BaseModel):
    incident_id: str
    detected_at_utc: datetime
    deploy_id: str
    invariant_id: Literal['INV-01']
    original_namespace: str
    original_intent_id: str
    evidence_hash: str

class IncidentCapsule(BaseModel):
    event: IncidentEvent
    pre_transaction_fixture_sha256: str
    bad_release_sha: str
    prior_release_sha: str
    changed_files: list[str]
    fault_plan_id: str
    allowed_patch_path: Literal['demo_store/domain/payment_retry.py']

class Scenario(BaseModel):
    id: Literal['S01', 'S02', 'S03', 'S04', 'S05', 'S06', 'S07', 'S08']
    executor: Literal['UI', 'API']
    fault: Literal['NONE', 'DROP_AFTER_CAPTURE_ONCE',
                  'TIMEOUT_BEFORE_CAPTURE_ONCE']
    required_invariants: list[str]
    fixture_fields: dict[str, str] = Field(default_factory=dict)
    deadline_seconds: int = 25

class ScenarioResult(BaseModel):
    scenario_id: str
    candidate_id: str
    namespace: str
    status: Literal['PASS', 'FAIL', 'TIMEOUT', 'ABSTAIN', 'NOT_RUN']
    browser_mode: Literal['JEV', 'PLAYWRIGHT', 'API']
    code_sha256: str
    seed_sha256: str
    oracle_version: str
    checks: list[InvariantResult]
    app_evidence_path: str
    provider_evidence_path: str
    trace_path: str | None
    elapsed_ms: int

class CandidateEvaluation(BaseModel):
    candidate_id: str
    rank: int
    code_sha256: str
    build_passed: bool
    bad_signature_removed: bool
```

```

scenarios: list[ScenarioResult]
status: Literal['PASS', 'FAIL', 'IN_PROGRESS', 'SKIPPED_GENERATION']
failure_reasons: list[str] = Field(default_factory=list)

```

```

class RepairLease(BaseModel):
    lease_id: str
    incident_id: str
    candidate_id: str
    candidate_sha256: str
    scope: Literal['synthetic_checkout_router']
    issued_at_utc: datetime
    expires_at_utc: datetime
    evidence_bundle_sha256: str
    status: Literal['ACTIVE', 'SAFE_HOLD', 'RENEWAL_REQUIRED',
                   'SUPERSEDED', 'STOPPED']
    renewal_count: int = 0

```

InvariantResult includes id, passed, expected, observed and structured evidence IDs. StageEvent includes monotonic sequence, UTC timestamp, incident ID, stage, action, counters and current handler. RepairReceipt includes both raw evidence hashes and human-readable explanations, already_affected_orders, newly_prevented_orders (only observed counted values), impact_source, candidate_statuses, model/browser versions and measured stage timings. No receipt number is fabricated for narrative effect.

Incident statuses: DETECTED -> SAFE_HOLD -> REPRODUCING -> VALIDATING -> SELECTING -> STAGED_SMOKE -> LEASED, with ESCALATED from any failing gate. Contextualization is a timed **event/task** overlapping reproduction and validation, not a blocking state transition; it does not block deterministic A/B. Each transition is an atomic compare-and-swap on (incident_id, state, version) in control-plane SQLite; repeated POST /run with same incident ID returns current run, not a second race. Persist event before SSE broadcast. A system restart reloads state and leaves router in SAFE_HOLD until evaluations and lease are revalidated. Gemini/jev cannot set state directly.

HTTP API:

- POST /api/demo/reset: test-only; clear controlled demo instances, not original evidence until operator chooses a new demo run.
- POST /api/demo/deploy-bug: test-only deployment pointer and flag.
- POST /api/demo/trigger-checkout: create one original synthetic intent and post-capture lost response; returns incident ID.
- POST /api/incidents/{id}/run: idempotent orchestrator start.
- POST /api/incidents/{id}/safe-stop: immediately stop automation and hold known safe router behavior; cannot reenale bad retry.
- GET /api/incidents/{id}: persisted status and measured timing.
- GET /api/incidents/{id}/events: SSE with Last-Event-ID replay.
- GET /api/incidents/{id}/receipt: only a final or explicitly partial receipt.
- GET /api/worlds/{id}: status and hashes; no provider evaluator credentials.
- GET /api/health: normal health remains green during semantic bug.

Admin/demo mutation endpoints require a local operator token; the button and router are outside model control. A control-plane readback after any lease actuation must confirm the chosen handler hash, scope and router generation. If readback differs, enter SAFE_HOLD/ESCALATED.

9.1 Lease, staged checks and safe stop

After candidate pass, the router directs **synthetic** checkout intents to the chosen handler at stages 5%, 25%, 100% by a stable hash of intent ID and configured seed. Each stage runs at least 10 new synthetic paid-intent probes plus two lost-response probes; all must satisfy the declared invariants. Other synthetic traffic remains on the SAFE_HOLD handler, which intentionally blocks new capture submissions, **not on the known bad retry**. Count and show numerator/denominator and control cohort results; the control group is expected to have zero captures and degraded availability, so it is a safety reference rather than a same-availability baseline. These small samples are functional smoke checks, not statistical safety or p95 latency estimates. The [Google SRE canary guidance](#) concerns controlled partial rollout with meaningful evaluation; the demo is labeled accordingly.

Lease issuance is an atomic transaction that writes (candidate_hash, scope, evidence_hash, expiry, router_generation) and then changes router target only after readback. The guardian probes every 15 seconds with synthetic traffic and watches trusted ledger invariants; a failure routes to SAFE_HOLD, records it and pages operator. At TTL, transition ACTIVE -> RENEWAL_REQUIRED; retain last safe routing only if guardian remains green, otherwise SAFE_HOLD. A human must authorize renewal in real use; the demo may simulate one bounded renewal and label it. The **Safe Stop** button cancels new world work and staged changes, preserves the last verified safe handler or enters SAFE_HOLD, and records an operator handoff. No generic “revert” step may restore the proven bad path.

10. Exact build plan and repository contract

Use a single Python 3.12 environment managed by uv; pin committed dependency versions and the Jev model used in final calibration. Serve a prebuilt React/Vite dashboard/storefront statically from FastAPI. Do not rebuild frontend in candidate worlds. Keep provider and evaluator packages outside patchable world image and credentials outside generated patch context.

```
nightwatch/
  README.md, AGENTS.md, pyproject.toml, uv.lock, .env.example, Makefile
  apps/control_plane/
    main.py, config.py, contracts.py, state.py, events.py, routes.py
    orchestrator.py, router.py, lease.py, receipt.py
    adapters/{conduct.py, gemini.py, world.py, modal_world.py,
              local_world.py, browser.py, jev_executor.py}
    services/{detector.py, capsule.py, candidate_factory.py,
              scenario_registry.py, scheduler.py, oracle.py, canary.py}
  apps/demo_store/
    app.py, domain/{models.py, db.py, checkout.py, payment_retry.py,
                    safe_handler.py, provider_client.py, refunds.py, inventory.py}
    static/ # built storefront, variant X/Y selected by fixture
  apps/trusted_provider/
    app.py, ledger.py, fault_plan.py, auth.py # separate service/store
  apps/dashboard/src/
  fixtures/{pre_transaction_seed.sqlite, incident.json,
            scenarios.json, impact_graph_fallback.json}
  tests/{unit,integration,isolation,fault_injection}/
  scripts/{smoke_all.py, seed.py, reproduce.py, run_demo.py,
           reset_demo.py, benchmark_browser.py, verify_receipt.py}
```

Implementation order: dependency graph, not visual polish order:

1. Create trusted provider ledger, status inquiry, fault plans and immutable intent/order schema. Write the lost-response test: bad release duplicates, safe handler confirms one capture, pre-capture timeout is pending. This test makes the central demo logically possible.
2. Implement pre-transaction fixtures, independent oracle and all eight scenario contracts. Run the two negative controls. Freeze oracle source and hash before Gemini integration.
3. Implement control-plane state store, router SAFE_HOLD, selector, lease and SSE. Use an in-process local world adapter first; complete run_demo.py without Gemini/Jev/Modal using deterministic fixtures and Playwright. This is a meaningful fallback and catches payment logic before distributed debugging.
4. Implement Modal worlds behind WorldFactory, prebuilt image, Sandbox Connect Tokens or authorized proxy, readiness, teardown and namespace reachability. Verify A/B/C run against the same frozen fixture with separate provider partitions.
5. Implement JevExecutor, two UI variants, three isolated browser processes, freshness checks, calibrated head-level gate and actual concurrency smoke. Mark every fallback in the result.
6. Add Gemini patch generator with exact one-file allowlist; add Conduct real adapter **only using the sponsor's provided contract**. Keep static graph explicit. Rehearse missing integration modes.
7. Implement staged synthetic router checks, guardian, receipt and dashboard; rehearse from real trigger twice. Improve visual polish after every safety/evidence gate works.

The README must give exact setup commands, env names, provider endpoint configuration,

run/test commands, known limitations, how to supply sponsor docs, and a three-minute demo script. AGENTS.md must repeat the trust boundaries, forbid edits to evaluator/provider fixtures by generated candidate, and state that an uncertain outcome remains uncertain. Do not make an implementer reverse engineer a diagram for the run sequence.

10.1 Adapter contracts and bounded configuration

```
class WorldFactory(Protocol):
    async def create(self, *, incident_id: str, candidate: Candidate,
                    seed_sha256: str) -> WorldHandle: ...
    async def reset_scenario(self, *, world: WorldHandle,
                           scenario: Scenario, namespace: str) -> None: ...
    async def close(self, world: WorldHandle) -> None: ...

class BrowserExecutor(Protocol):
    async def run(self, *, world: WorldHandle, scenario: Scenario,
                 namespace: str) -> BrowserTrace: ...

class ImpactAdapter(Protocol):
    async def affected_processes(self, *, changed_files: list[str],
                               diff: str) -> ImpactGraph: ...

class EvidenceOracle(Protocol):
    async def evaluate(self, *, world: WorldHandle, scenario: Scenario,
                     namespace: str) -> list[InvariantResult]: ...
```

WorldHandle holds candidate_id, sandbox_id, authenticated URL access handle, app code hash, seed hash, readiness time and cleanup handle. Its token is never serialized to dashboard or receipt. ImpactGraph.source is either CONDUCT_LIVE or STATIC_FALLBACK, with an evidence ID and version. BrowserTrace includes observed snapshot/action IDs, operation/target margins, stale discards, Jev call count, fallback reason, status and wall-clock/monotonic timings. The oracle gets its own provider credential and never accepts candidate_evaluation from the candidate process.

.env.example declares (empty credentials, safe defaults):

```
GOOGLE_API_KEY=
GEMINI_MODEL=gemini-3.8-flash
TYPESAFE_API_KEY=
TYPESAFE_MODEL=jev-1.13.0
MODAL_TOKEN_ID=
MODAL_TOKEN_SECRET=
CONDUCT_ENABLED=0
CONDUCT_API_KEY=
CONDUCT_BASE_URL=
WORLD_EXECUTOR=modal
BROWSER_EXECUTOR=jev
MAX_BROWSER_WORKERS=3
MAX_STEPS_PER_UI_SCENARIO=20
UI_SCENARIO_TIMEOUT_SECONDS=25
INCIDENT_DEPLOY_WINDOW_SECONDS=900
LEASE_SECONDS=7200
DEMO_MODE=1
```

Google's [model list](#) currently names gemini-3.8-flash; Pydantic AI supports Google providers and thinking settings. Verify the configured model and actual key/quota with a live structured-output smoke before committing to it. TypeSafe Jev is early access: access/quotas must be confirmed separately; no key is silently assumed. There is no OpenRouter text helper in the fixed-fixture critical path. If live Conduct docs lack an endpoint, use STATIC_FALLBACK. Do not guess endpoint paths, method names, response fields or quota for sponsor services.

10.2 Control-plane orchestration pseudocode

```

async def run_incident(event: IncidentEvent) -> RunResult:
    async with incident_lock(event.incident_id):
        capsule = freeze_capsule(event) # immutable seed hash + original evidence
        assert qualifying_recent_deploy(capsule)
        await router.enter_safe_hold(event.incident_id) # preauthorized action
        save_event('SAFE_HOLD', router.readback())
        bad = await replay_bad_release(capsule, scenario='S02')
        if bad.signature != EXPECTED_DOUBLE_CAPTURE:
            return escalate('INCIDENT_NOT_REPRODUCED', bad.evidence)
        impact_task = create_task(impact_adapter.affected_processes(...))
        plan_task = create_task(gemini_plan_or_failure(capsule))
        a_task = create_task(build_world(rollback_candidate(capsule)))
        b_task = create_task(build_world(safe_handler_candidate(capsule)))
        # C begins after validated patch; it never blocks A/B creation.
        c_task = create_task(build_patch_world_when_ready(plan_task))
        evaluations = evaluate_worlds_with_serial_scenarios(
            [a_task, b_task, c_task], mandatory_scenarios=S01_T0_S08,
            independent_oracle=oracle,
        )
        # A passing rank-0 B can progress; otherwise await all possible winners.
        winner = await first_policy_complete_eligible(evaluations)
        if winner is None: return escalate('NO_SAFE_CANDIDATE')
        if not await staged_synthetic_smoke(winner):
            return escalate_preserving_safe_hold('ROLLOUT_CHECK_FAILED')
        lease = await issue_lease_atomically(winner, evidence_hash())
        assert router.readback().candidate_sha256 == winner.code_sha256
        persist_receipt(
            lease, evaluations,
            original_affected_order=event.original_intent_id,
        )
        return RunResult(lease=lease)

```

Every task has a cleanup finally; world teardown or expiry is idempotent. An interrupted execution restores persisted state and SAFE_HOLD, not an old buggy flag. If `first_policy_complete_eligible` returns B while C still runs, receipt marks C IN_PROGRESS; the final receipt revision updates C status without changing lease unless guardian/policy requires it. If B fails, A cannot be chosen until every lower-rank candidate has definitively failed; C may be selected only after B and A fail. C unavailable cannot block B/A. Candidate verdicts are based on exact mandatory scenario IDs and checks, not just `all([])` on an empty list.

11. Acceptance tests, preflight gates and failure modes

11.1 Tests that protect the thesis

- `test_post_capture_timeout`: bad branch creates two captures; safe handler produces PAID and one capture after inquiry; patch produces no duplicate; original ledger is unaffected by worlds.
- `test_pre_capture_timeout`: safe handler and patch show PENDING, zero captures and no confirmation; provider read failure also stays pending.
- `test_provider_key_semantics`: same key/same frozen parameters returns prior result; same key/different amount or currency is rejected; fresh key in bad branch can append another capture.
- `test_double_click_race`: two simultaneous requests with same checkout intent produce one order/capture/confirmation; two orders with one capture each fails INV-01.
- `test_refund`: full and partial refunds preserve historical capture, enforce upper bound and refund-intent idempotency.
- `test_negative_controls`: fake UI success without capture and two orders for one intent both FAIL despite HTTP 200.
- `test_candidate_isolation`: simultaneous A/B/C S02 faults yield one independent fault con-

sumption each; browser contexts have no shared cookie/cart; seed hashes match; original ledger unchanged.

- `test_patch_guard`: patch touching provider/evaluator/config/tests, symlink, binary diff or >40 changed lines is rejected; malformed Gemini output does not block A/B.
- `test_jevstaleness`: changed page/target, low margin, invalid target, timeout, empty initial snapshot and uncertain submit all reobserve/abstain or fail; never blindly resubmit.
- `test_selector`: B expected only if all eight pass; B failure permits A; all fail produces no lease; empty/missing result cannot PASS; candidate hash mismatch fails.
- `test_stop_and_expiry`: `SAFE_STOP`, guardian failure and TTL never route to the known bad retry; guardian/lease/routing state survives a process restart.
- `test_replay_and_sse`: duplicate run idempotent; event replay after disconnect; original affected order included in receipt; no credentials in trace/UI.

Run `uv run pytest tests/unit tests/integration tests/isolation tests/fault_injection` and `uv run python scripts/run_demo.py --verify` after the final material change. The full demo must pass **twice consecutively**. A test may be simulated only when explicitly labeled, never passed using recorded traces presented as live.

11.2 Day-of smoke gate

Before polishing, verify: Gemini key/model + typed patch-plan response; Jev key/model + one valid Choice and per-head margins; browser profile separation and simultaneous two-world actions; trusted provider reachable from each candidate world; Modal world creation/readiness/token/auth/cleanup; A and B each pass S02; the independent negative-control test fails as expected; optional Conduct availability and source label. If Jev early access is missing, the project can show a **reduced deterministic Playwright demo** but must not claim Jev integration or complete the primary Jev acceptance gate.

11.3 Failure policy (no silent green state)

Failure	Required outcome
No plausible recent deploy, unknown invariant, or non-replayable incident	<code>SAFE_HOLD</code> if already triggered by known detector; <code>ESCALATE</code> with unsupported reason, no lease
Original bug does not reproduce under frozen fixture	<code>ESCALATE</code> ; do not generate a purported verified fix
Fake provider/evaluator unreachable or namespace/hash mismatch	Candidate/world FAIL; if all fail, <code>SAFE_HOLD</code> + <code>ESCALATE</code>
Conduct unavailable	<code>STATIC_FALLBACK</code> graph and explicit badge; A/B proceed
Gemini unavailable/malformed	<code>C_SKIPPED_GENERATION</code> ; A/B proceed; full three-candidate claim unavailable
Jev missing, rate limited, low margin or broken browser	Scenario <code>ABSTAIN</code> or clearly marked Playwright fallback; no Jev win or fake timing
Modal unavailable	Use pretested local Docker worlds with isolated provider; label mode; no surprise day-of implementation
Scenario timeout, missing check, app 200 without provider capture	Candidate FAIL, not partial PASS
Staged smoke/guardian failure or Safe Stop	Freeze or enter <code>SAFE_HOLD</code> ; never route back to bad retry; record operator handoff
>180 s elapsed	Keep truthful clock and safe state; issue lease later only if all checks pass and report actual time

11.4 Definition of done and fallback levels

Full primary demo: exactly three independent candidate worlds; eight mandatory scenarios attempted and evaluated per candidate unless an early failure conclusively rejects it (receipt must

show skipped cases and failure reason); two Jev UI journeys for every candidate that reaches UI tests; actual peak of at least two simultaneous browser workers and isolated contexts; original duplicate reproduced and shown separately; independent provider ledger and negative control; policy-selected mitigation; synthetic staged smoke; lease/readback; receipt; Safe Stop tested; two clean end-to-end rehearsals. If C is generated too slowly, a rank-0 B lease may be issued with C marked IN_PROGRESS, and full three-candidate comparison is claimed only after C finishes. Mandatory checks for the selected candidate cannot be skipped.

Reduced but truthful demo: A/B compare with static Conduct graph or local Docker worlds, or deterministic Playwright browser fallback. It may still show the payment/oracle/lease architecture but must label which integration or parallelism was absent. No “under three minutes with Jev and three futures” claim unless that exact path ran and was measured.

12. Demo presentation and what the result means

1. Open: “The site is healthy by HTTP checks, but one customer was charged twice.” Trigger the fake deployment and checkout. Show the trusted provider ledger: two captures, distinct keys, same intent; show /health=200.
2. Show immediate SAFE_HOLD on affected future intents. Explain this was preauthorized; it does not undo the original duplicate.
3. Show bad replay on frozen pre-transaction fixture, then A/B/C worlds with code/fixture hashes and distinct provider namespaces. Display simultaneous worker count rather than an animation suggesting unmeasured 24-way parallelism.
4. Show Jev on semantic variant Y selecting observed actions with operation and target margins. Run real provider oracle; show the negative-control test fails in a labeled developer panel if there is time.
5. Show each actual candidate outcome. If C fails or remains in progress, say so. Explain why deterministic policy chooses the lowest-risk **passing** option.
6. Show synthetic staged rollout smoke probes and lease/readback. End on receipt, original harmed intent requiring manual reconciliation, and the exact elapsed time. Display comparison against scripted Playwright; if Playwright wins on fixed variant X, acknowledge it and point to variant Y/unknown layouts as Jev’s target case.

Judge answer on product viability: a real customer would start with invariant packs and shadow/release verification, then opt into a preapproved safe hold and feature-flag registry. They would never grant a generated patch arbitrary production authority. The hackathon’s controlled money stub demonstrates architecture and correctness checks, not a real payment certification.

Judge answer on causality: matched bad-versus-candidate replay shows the candidate prevents this specific failure with this fixture. It does not prove the sole historical cause or guarantee behavior on untested users. The receipt carries limits and hashes so the evidence can be reproduced.

Judge answer on Jev: Jev offers fast bounded action/target decisions for changing UI journeys; its typed output cannot make arbitrary commands, but it can still choose the wrong legal action. The freshness guard and independent payment oracle are separate safeguards. [TypeSafe](#) publishes task-dependent latency claims; present local observed times and success rates, not general speedups.

13. Source notes and verification limits

Checked on 19 September 2026. Product documentation and service availability can change before event time; re-run smoke tests then. Sources support the stated capabilities and cautions; they do not prove this unbuilt NightWatch architecture will achieve its target latency.

- [TypeSafe introduction to Jev](#): early access, typed decisions, author-reported latency range, limitations of published comparisons.
- [TypeSafe Python SDK](#) and [speculative fan-out pattern](#): async client and atomic Choice questions; consult installed SDK for exact field names.
- [Pydantic AI TypeSafe guide](#): confidence is a per-field margin rather than correctness probability; threshold calibration and version pinning; option limits.
- [Browser Use jev-ultrafast](#) and [performance methodology](#): synchronous single-browser reference, known DOM/profile limits and narrow timing boundaries. [Browser Harness](#) distinguishes local browser use from many-browser Cloud usage.

- [Modal Sandbox reference](#) and [network access controls](#): readiness, sandbox lifecycle and authenticated Connect Tokens. An encrypted port alone is not an auth scheme.
- [Stripe idempotent requests](#), [advanced network-error handling](#) and [webhooks](#): same-key retries, unknown outcomes and reconciliation concepts. NightWatch's provider protocol is intentionally synthetic.
- [SQLite Online Backup API](#): consistent database snapshots.
- [Google SRE canarying](#): staged exposure and evaluation; NightWatch's tiny synthetic stages are functional smoke checks only.
- [Google Gemini models](#) and [Pydantic Google model integration](#): available model name and structured model client. Verify live key/quota.
- [Conduct product context](#) and [Conduct sandbox engineering](#): business-process mapping and isolated environments, not a public stable hackathon API contract.
- [Rootly AI SRE](#), [incident.io Investigations](#), [Datadog Bits AI](#), [LaunchDarkly guarded rollouts](#): adjacent product positioning; avoid first-ever claims.

Final instruction to coding agents: deliver the smallest complete, honest loop. Keep the provider ledger and evaluator outside candidate control. Never declare a payment settled from a timeout or a model assertion. Never reenable a known duplicate-charge path via panic, TTL or failed rollout. Show actual timings, actual concurrency, actual model calls and every fallback in the receipt.